

Llenguatges de Programació, FIB, 14 de juny de 2024

L'examen dura tres hores. Poseu el vostre nom a cada full. Contesteu cada problema en fulls diferents. Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes. Feu bona lletra.

1 λ -càlcul

(2 punts)

Recordeu que en λ -càlcul tenim aquestes equivalències:

true	$\equiv$	$\lambda x.\lambda y.x$
false	$\equiv$	$\lambda x.\lambda y.y$
0	$\equiv$	$\lambda s.\lambda z.z$
1	$\equiv$	$\lambda s.\lambda z.sz$
2	$\equiv$	$\lambda s.\lambda z.s(sz)$
if	$\equiv$	$\lambda c.\lambda x.\lambda y.cxy$
leq	$\equiv$	$\lambda m.\lambda n.\text{isZero}(\text{minus } m\ n)$
isZero	$\equiv$	$\lambda n.n(\lambda x.\text{false})\text{true}$
minus	$\equiv$	$\lambda m.\lambda n.n\ \text{pred } m$

Per al cas de **pred**, utilitzarem la notació aritmètica per simplificar i escurçar la solució del problema. És a dir, assumirem que **pred 2** $\equiv$ **1** i que **pred 0** $\equiv$ **0**.

1. Digueu com faríeu el **maxim** de dos nombres naturals utilitzant els combinadors anteriors.
2. Comproveu el resultat d'avaluar **maxim 2 1** en ordre normal tot explicitant cada operacions que apliqueu.

2 Avaluació mandrosa

(2 punts)

Considereu aquesta seqüència infinita definida recursivament:

$$rec = 0 : \text{scanl } (+) \ 1 \ rec \quad \text{-- (R)}$$

Aquestes són unes possibles definicions per a les funcions auxiliars que fa servir:

$$\text{scanl } _ _ [] = [] \quad \text{-- (S1)}$$

$$\text{scanl } f \ z \ (x:xs) = z : \text{scanl } f \ (f \ z \ x) \ xs \quad \text{-- (S2)}$$

$$\text{take } 0 _ = [] \quad \text{-- (T1)}$$

$$\text{take } n \ (x:xs) = x : \text{take } (n-1) \ xs \quad \text{-- (T2)}$$

Contesteu les preguntes següents:

1. Mostreu (pas a pas) com Haskell avalua **take 3 rec** utilitzant avaluació mandrosa. Per fer-ho més curt, apliqueu les sumes ansiosament. A cada pas, indiqueu quina equació s'aplica. Podeu començar així:

$$\begin{aligned} \text{take } 3 \ rec &= \\ &= \text{take } 3 \ (0 : \text{scanl } (+) \ 1 \ rec) \quad \text{-- (R)} \\ &= \dots \end{aligned}$$

2. Què calcula *rec*?
3. Quin és el temps asimptòtic de l'avaluació de **take n rec**? Justifiqueu la vostra resposta.
4. Doneu una definició de la funció *rec* més eficient i digueu quin és el temps asimptòtic de **take n rec** amb la vostra definició.

3 Inferència de tipus

(2 punts)

Recordeu els tipus d'aquests operadors habituals:

```
(:) :: a -> [a] -> [a]
fmap :: Functor f => (a -> b) -> f a -> f b
<*> :: Applicative f => f (a -> b) -> f a -> f b
```

A més, recordeu la definició de la classe **Applicative** i algunes de les seves instàncies:

```
class Functor f => Applicative f where
  pure :: a -> f a
  <*> :: f (a -> b) -> f a -> f b
```

```
instance Applicative Maybe where
  pure = Just
  Just f <*> Just x = Just (f x)
  _ <*> _ = Nothing
```

```
instance Applicative [ ] where
  pure x = [x]
  fs <*> xs = [f x | f <- fs, x <- xs]
```

Haskell conté una classe **Traversable** a la qual pertanyen totes les estructures de dades que poden ser recorregudes d'esquerra a dreta. Una de les seves funcions bàsiques és *sequenceA*:

```
instance Traversable Maybe where
  sequenceA Nothing = Nothing
  sequenceA (Just x) = fmap Just x

instance Traversable [ ] where
  sequenceA [] = pure []
  sequenceA (u:us) = (fmap (:) u) <*> (sequenceA us)
```

1. Calculeu (pas a pas) quan valen aquestes expressions:

- *sequenceA* [Just 1, Just 2]
- *sequenceA* [Just 1, Nothing]
- *sequenceA* (Just [1, 2])
- *sequenceA* [[1, 2], [3, 4]]

2. Inferiu el tipus complet de:

```
sequenceA (u:us)
```

4 Clausures en Python

(2 punts)

Una clausura és una funció que té accés a l'àmbit que la conté. A continuació teniu un exemple del seu ús en Python:

```
def exterior ( inici ):
    fi = '!'
    def interior (nom):
        print( inici , nom, fi )
    return interior

saluda = exterior ( 'Hola' )
saluda( 'Pep' )
```

Aquest codi imprimeix Hola Pep !.

També, en Python podem crear una funció que tingui un nombre indeterminat de paràmetres amb l'operador * de la forma següent:

```
def aux(*args ):
    print(type(args))
    print(args)
aux(1,2)
```

Aquest codi imprimeix <class 'tuple'> i (1, 2).

La llibreria *functools* conté una funció *partial* relacionada amb la currificació.

1. Doneu una implementació python de la funció *partial* que faci funcionar el codi següent fent que la sortida sigui 6:

```
multiplica = lambda x,y: x * y
doble = partial ( multiplica , 2)
print(doble (3))
```

2. Doneu una implementació de la funció *partialN* que faci funcionar el codi següent fent que la sortida sigui respectivament 10 i 25:

```
from functools import reduce

def sumaRed(*args):
    return reduce(suma, list(args), 0)

print(sumaRed(1,2,3,4))
sumaRed10 = partialN(sumaRed, 1, 2, 3, 4)
print(sumaRed10(5, 5, 5))
```

5 Una IA m'explica Clojure i em posa un problema (2 punts)



Bon dia!



Bon dia, amb què et puc ajudar?



Voldria tenir alguna noció de Clojure.



Excel·lent! Clojure és un llenguatge de programació funcional basat en Lisp, dissenyat per ser simple i expressiu i que compila a la Java Virtual Machine.



Dona'm alguns exemples senzills de Clojure tenint en compte que ja sé Haskell.



I tant. Sabent Haskell no et costarà gens. Però tingues en compte que cal posar parèntesis al voltant de cada expressió. A continuació tens alguns exemples d'expressions en Clojure amb la sortida després d'un comentari (punt i coma):

```
(zero? 1)           ; false
(dec 67)             ; 66
(mod 7 2)            ; 1
(if (< 7 2) true false) ; false
;; funció anònima
((fn [x y] (+ x y)) 2 3) ; 5
;; let
(let [x 3] (+ x 2))    ; 5
```



Com es definiria la funció factorial en Clojure?



La funció factorial s'escriu naturalment de forma recursiva en Clojure:

```
(defn factorial [n]
  (if (zero? n)
      1
      (* n (factorial (dec n)))))
```



Però... amb un nombre molt gran, no hi ha perill de sobreiximent de la pila?



En efecte, en treballar amb nombres grans, aquesta funció té problemes amb l'espai de la pila. Una possible solució consistiria a utilitzar l'optimització de la recursivitat per la cua (*tail call optimization*), però la JVM no la suporta directament. Tanmateix, Clojure ofereix una funció **recur** per poder-la aplicar: Quan s'utilitza la funció **recur** al final de la funció recursiva, el compilador activa l'optimització, que fa que no s'incrementi la mida de la pila:

```
(defn factorial2 [n acc]
  (if (zero? n)
      acc
      (recur (dec n) (* n acc))))
```

```
(defn factorial [n] (factorial2 n 1))
```



Hauria de mirar-ho bé! Quin exercici em proposaries per practicar?



Un exercici senzill seria escriure una funció **prime?** en Clojure que indiqui si un nombre natural és un nombre primer o no. Assegura't d'utilitzar la funció **recur** per evitar sobreiximents de la pila. Fixa't que no et cal saber com es calculen arrels quadrades en Clojure.